

Diseño e implementación de la interfaz gráfica para una consola retro embebida

Cesar Romualdo Ramírez Cervantes
Fundamentos de Sistemas Embebidos

1. Objetivo

Diseñar e implementar la interfaz gráfica principal de una consola retro embebida basada en Raspberry Pi OS Lite, de manera que el usuario pueda visualizar la biblioteca de juegos, navegar mediante gamepad, seleccionar ROMS compatibles y recibir mensajes visuales sobre el estado del sistema sin utilizar escritorio gráfico, teclado ni mouse durante la operación final.

El presente reporte se enfoca en el módulo de interfaz de usuario, desarrollado principalmente en el archivo `src/menu.py`. Aunque el proyecto completo integra emulación, arranque automático, sincronización USB, servicios del sistema operativo y reglas de detección de dispositivos, la parte documentada aquí corresponde al menú gráfico, al manejo de eventos del control y a la forma en que la interfaz comunica la selección de juegos al resto del sistema.

2. Introducción

El proyecto final consistió en el desarrollo de una consola retro embebida utilizando Raspberry Pi OS Lite, Python, Pygame, Mednafen, scripts Bash, systemd y udev. La función del sistema es arrancar directamente a una interfaz gráfica propia, mostrar una galería de juegos, permitir la selección mediante gamepad y ejecutar ROMS de NES, SNES y Game Boy Advance mediante un emulador.

De acuerdo con Barr, un sistema embebido es una combinación de hardware y software diseñada para realizar una función específica [1]. Esta definición permite distinguir un sistema embebido de una computadora de propósito general. Una computadora personal puede ejecutar múltiples tareas definidas por el usuario; en cambio, un sistema embebido se configura para cumplir una función concreta dentro de un producto o dispositivo.

En este proyecto, la Raspberry Pi no se utiliza como computadora de escritorio, sino como consola dedicada. La tarjeta se configura para iniciar automáticamente, presentar una interfaz de usuario en pantalla completa y recibir entradas desde un gamepad. Por tanto, la interfaz gráfica no es un complemento secundario, sino el punto central de operación del sistema.

La interfaz debía cumplir varios requisitos: ser clara, funcionar sin escritorio, permitir operación completa mediante gamepad, mostrar ROMS disponibles, actualizar la lista cuando se agregaran juegos y comunicar la selección al emulador. Para lograrlo se utilizó Pygame, ya que permite dibujar una interfaz propia y procesar eventos de joystick desde Python.

3. Antecedentes

3.1. Sistemas embebidos

Un sistema embebido suele estar condicionado por el hardware disponible, las entradas y salidas físicas, la memoria, los periféricos y las restricciones de operación. Barr señala que los sistemas embebidos contienen algún tipo de entrada y salida, y que dichas salidas normalmente dependen de las entradas y del estado actual del sistema [1]. En este proyecto, las entradas principales son el gamepad y la memoria USB; las salidas principales son la pantalla HDMI, los mensajes gráficos del menú y las señales de audio.

Otra característica relevante es que el software embebido normalmente debe permanecer en ejecución mientras el dispositivo esté encendido. Barr explica que muchos programas embebidos se estructuran alrededor de ciclos de ejecución continua, ya que la función del software no termina después de realizar una sola operación [1]. Esto se refleja en la interfaz de la consola: el menú permanece activo, procesa eventos del control, actualiza la pantalla y espera acciones del usuario.

Aunque la Raspberry Pi ejecuta un sistema operativo Linux, el comportamiento final se restringe a una función específica. El usuario no debe interactuar con terminales, ventanas o aplicaciones generales, sino únicamente con la interfaz de la consola.

3.2. Interfaz gráfica en un sistema embebido

En una computadora convencional, una aplicación gráfica suele ejecutarse dentro de un entorno de escritorio. En este proyecto se evitó ese enfoque porque el requisito del prototipo exige no utilizar escritorio Linux ni manejadores de ventanas superpuestas. Por ello se eligió una interfaz en pantalla completa ejecutada directamente desde el sistema.

La interfaz fue desarrollada con Pygame. Esta biblioteca permite crear superficies de dibujo, renderizar texto, manejar imágenes, reproducir audio y procesar eventos de teclado, mouse o joystick. Para el caso de este proyecto, lo más importante fue el soporte para joystick y la posibilidad de dibujar una interfaz propia sin depender de un entorno gráfico de escritorio.

3.3. Relación entre menú, orquestador y emulador

El módulo de interfaz no ejecuta la emulación directamente. En lugar de ello, el menú permite al usuario seleccionar una ROM y escribe la ruta correspondiente en un archivo temporal. Posteriormente, el script `start.sh` lee dicha ruta y ejecuta Mednafen.

Esta separación de responsabilidades fue una decisión de diseño importante. El menú se encarga de la interacción gráfica; el script Bash se encarga del flujo de ejecución; y Mednafen se encarga de la emulación. Al mantener estas responsabilidades separadas, el sistema resulta más fácil de probar,

modificar y explicar.

4. Lista de materiales

Para la implementación y prueba del sistema completo se consideraron los siguientes elementos:

- Raspberry Pi compatible con Raspberry Pi OS Lite.
- Tarjeta microSD para el sistema operativo.
- Fuente de alimentación adecuada para la Raspberry Pi.
- Pantalla HDMI.
- Gamepad o joystick USB.
- Memoria USB para importación de ROMS.
- Bocinas o salida de audio por HDMI.
- Conexión a red para clonar el repositorio y actualizar paquetes.

Aunque este reporte se enfoca en el software de interfaz, los periféricos son relevantes porque determinan la forma de interacción. La pantalla HDMI funciona como salida principal, mientras que el gamepad funciona como entrada principal del usuario.

5. Descripción de la tarjeta controladora

La Raspberry Pi actúa como tarjeta controladora del sistema. Sus funciones principales dentro del proyecto son ejecutar Raspberry Pi OS Lite, cargar los servicios de arranque, ejecutar el menú gráfico, leer entradas del gamepad, acceder al almacenamiento local de ROMS y ejecutar el emulador.

La elección de Raspberry Pi es adecuada para este proyecto porque integra procesador, memoria, salida HDMI, puertos USB y soporte para Linux. Esto permite conectar directamente los periféricos necesarios para la consola: pantalla, gamepad y memoria USB. Además, al utilizar Raspberry Pi OS Lite se evita cargar un escritorio completo, reduciendo componentes innecesarios para el propósito del sistema.

6. Advertencias de seguridad y cuidado del equipo

La Raspberry Pi debe alimentarse con una fuente adecuada para evitar reinicios inesperados o corrupción de datos en la microSD. También debe apagarse correctamente, ya que cortar la alimentación durante operaciones de escritura puede dañar archivos del sistema o de la biblioteca de juegos.

La tarjeta microSD y las memorias USB deben manipularse con cuidado. Se recomienda desmontar los dispositivos antes de retirarlos cuando se realicen pruebas manuales. En el flujo automatizado del proyecto, el script de sincronización desmonta la USB después de copiar ROMS, pero durante pruebas de desarrollo se debe verificar que la operación haya terminado.

También debe evitarse tocar los contactos metálicos de la microSD o de la Raspberry Pi con las manos sucias o húmedas. La acumulación de grasa, polvo o humedad puede generar fallas intermitentes. Además, se debe mantener ventilación suficiente, especialmente durante la emulación, ya que ésta puede aumentar la carga del procesador.

7. Configuración general del sistema

El proyecto está organizado en módulos. La interfaz gráfica se encuentra en `src/menu.py`; el arranque del menú y del emulador se coordina mediante `scripts/start.sh`; la sincronización de ROMS desde USB se realiza con `scripts/sync_roms.sh`; el servicio principal se define en `systemd/retromenu.service`; y la detección USB se configura en `udev/99-usb-sync.rules`.

La interfaz usa la ruta local mostrada en el Código 1.

```
/home/pi/roms
```

Código 1: Directorio local donde la interfaz busca las ROMS.

En esa carpeta se almacenan las ROMS disponibles para el usuario. El menú filtra los archivos según su extensión, por lo que sólo se muestran juegos compatibles con las consolas requeridas.

8. Desarrollo de la interfaz gráfica

8.1. Archivo principal de la interfaz

La parte de interfaz se desarrolló en el archivo mostrado en el Código 2.

```
src/menu.py
```

Código 2: Archivo principal del módulo de interfaz.

Este archivo concentra la configuración de Pygame, las rutas de recursos, la paleta de colores, el escaneo de ROMS, el dibujo del menú, el manejo de eventos del gamepad y la comunicación con el orquestador del sistema.

El Código 3 muestra parte de la configuración inicial utilizada por el menú.

```
1 ROM_DIR = "/home/pi/roms"
2 EMULATOR = "mednafen"
3 FONT_FILE = "/home/pi/retro_font.ttf"
4 AUDIO_USB = "/home/pi/assets/moneda.wav"
5
6 os.environ["SDL_VIDEODRIVER"] = "kmsdrm"
```

Código 3: Configuración principal de rutas y controlador gráfico en `menu.py`.

La ruta `ROM_DIR` indica dónde se buscarán los juegos. La ruta `FONT_FILE` permite cargar una fuente personalizada para el menú. La ruta `AUDIO_USB` se usa para reproducir un sonido cuando se detecta una acción relacionada con USB. Finalmente, `SDL_VIDEODRIVER=kmsdrm` permite que la interfaz se ejecute sin depender de un escritorio gráfico.

8.2. Decisión de usar Pygame

Se utilizó Pygame porque permite construir una interfaz gráfica propia con una cantidad razonable de código. La alternativa habría sido utilizar un entorno gráfico completo, una interfaz web o una distribución especializada, pero esas opciones no se ajustaban al objetivo del proyecto.

Usar Pygame permitió tomar control directo de los elementos de pantalla: fondo, título, lista de juegos, selección activa, mensajes de estado y ventanas emergentes. También permitió procesar eventos de joystick, lo cual era indispensable para operar el sistema como consola.

La decisión también favoreció la separación entre interfaz y emulación. Pygame sólo se encarga del menú, mientras que Mednafen se ejecuta por separado. Esto evita que la interfaz tenga que administrar internamente todos los detalles del emulador.

8.3. Pantalla completa sin escritorio

El menú se ejecuta en pantalla completa. Esta decisión responde al requisito de que la interfaz esté libre de ventanas superpuestas y no dependa de un escritorio. En una consola comercial, el usuario no ve una barra de tareas ni ventanas; ve directamente el menú del sistema. Se buscó replicar ese comportamiento.

El uso de `kmsdrm` permite que SDL, utilizado por Pygame, trabaje directamente sobre la salida gráfica. Esto reduce la dependencia de componentes de escritorio y ayuda a que la Raspberry Pi funcione como dispositivo dedicado.

8.4. Diseño visual del menú

El diseño visual se basó en una estética Synthwave. Se eligieron colores oscuros, tonos neón y una cuadrícula en perspectiva para reforzar la idea de consola retro. Esta decisión no es sólo estética: también ayuda a jerarquizar la información. El fondo no compite con la lista de juegos, mientras que el color de selección permite distinguir claramente el elemento activo.

El fondo se dibuja mediante una función específica. La lógica general consiste en crear un degradado vertical y posteriormente dibujar una cuadrícula de perspectiva. El Código 4 muestra la estructura de dicha función.

```
1 def draw_synthwave_background(screen, width, height):
2     for y in range(height):
3         ratio = y / height
4         r = int(COLOR_FONDOALTO[0] * (1 - ratio) +
5               COLOR_FONDOBASICO[0] * ratio)
6         g = int(COLOR_FONDOALTO[1] * (1 - ratio) +
7               COLOR_FONDOBASICO[1] * ratio)
8         b = int(COLOR_FONDOALTO[2] * (1 - ratio) +
9               COLOR_FONDOBASICO[2] * ratio)
10        pygame.draw.line(screen, (r, g, b), (0, y), (width, y))
```

Código 4: Función encargada de dibujar el fondo estilo Synthwave.

La decisión de generar el fondo desde código evita depender de una imagen externa para la pantalla principal. Esto facilita transportar el proyecto, ya que la interfaz puede reconstruirse a partir del código fuente.

8.5. Escaneo y filtrado de ROMS

El menú no contiene una lista fija de juegos. En cambio, escanea el directorio local de ROMS y filtra los archivos por extensión. Esta decisión permite que la biblioteca sea dinámica: si se agregan o eliminan juegos, el menú puede reflejarlo.

El Código 5 muestra la función que obtiene los juegos disponibles.

```
1 def obtener_juegos():
2     if not os.path.exists(ROM_DIR):
3         return []
4
5     return sorted([
6         f for f in os.listdir(ROM_DIR)
7         if f.endswith(('.nes', '.sfc', '.smc', '.gba'))
8     ])
```

Código 5: Escaneo de ROMS compatibles en el directorio local.

Las extensiones aceptadas corresponden a las consolas requeridas por el proyecto. La relación se resume en la Tabla 1.

Cuadro 1: Extensiones reconocidas por la interfaz.

Sistema	Extensiones
NES	.nes
SNES	.sfc, .smc
Game Boy Advance	.gba

El filtrado evita que aparezcan archivos no válidos en la interfaz. Esto es importante porque las memorias USB o el directorio local pueden contener archivos auxiliares, documentos o elementos que no deben presentarse como juegos.

8.6. Formato de nombres de juegos

Además de filtrar archivos, la interfaz mejora la forma en que los nombres se muestran al usuario. Los nombres de archivo suelen incluir guiones bajos, guiones medios o extensiones. Para que la lista sea más legible, se elimina la extensión y se reemplazan separadores por espacios.

El Código 6 muestra la función de formateo.

```
1 def formatear_nombre_juego(nombre_archivo):
2     nombre_sin_ext = nombre_archivo.rsplit('.', 1)[0]
3     nombre_limpio = nombre_sin_ext.replace('_', ' ').replace('-', ' ')
4     return nombre_limpio.title()
```

Código 6: Formateo de nombres de archivo para mostrarlos en la interfaz.

Esta decisión mejora la experiencia de usuario. En lugar de mostrar un archivo como `super_demo_nes.nes`, el menú puede mostrar un nombre más legible. Esto hace que la interfaz sea más cercana a una galería de juegos que a un explorador de archivos.

8.7. Lista desplazable

El proyecto exige contar con al menos quince ROMS. Si se intentara mostrar toda la biblioteca al mismo tiempo, la pantalla podría saturarse. Por ello, el menú trabaja con una lista desplazable: se mantiene un índice seleccionado y sólo se dibuja una porción visible de la lista.

La decisión de implementar desplazamiento permite que la interfaz siga siendo clara aunque la biblioteca crezca. El usuario puede navegar hacia arriba o hacia abajo con el gamepad y la pantalla se actualiza para mantener visible el juego seleccionado.

El Código 7 muestra una versión representativa de la lógica de lista desplazable.

```
1 max_visibles = 12
2 inicio_lista = 0
3
4 if len(juegos) > max_visibles:
5     inicio_lista = max(0, indice_seleccionado - (max_visibles // 2))
6
7     if inicio_lista > len(juegos) - max_visibles:
8         inicio_lista = len(juegos) - max_visibles
9
10 juegos_visibles = juegos[inicio_lista : inicio_lista + max_visibles]
```

Código 7: Lógica de desplazamiento para listas con más juegos que elementos visibles.

Esta lógica permite que la selección se mantenga dentro del rango visible y que la lista se desplace cuando el usuario navega por una biblioteca más grande.

8.8. Manejo de eventos del gamepad

El gamepad es la entrada principal del sistema. La navegación se realiza mediante eventos de Pygame. El evento JOYHATMOTION se usa para mover la selección, mientras que JOYBUTTONDOWN se usa para confirmar acciones.

El Código 8 representa la lógica de navegación mediante la cruceta o pad direccional.

```
1 if evento.type == pygame.JOYHATMOTION:
2     x, y = evento.value
3
4     if juegos:
5         if y == 1:
6             indice_seleccionado = (indice_seleccionado - 1) % len(juegos)
7         elif y == -1:
8             indice_seleccionado = (indice_seleccionado + 1) % len(juegos)
```

Código 8: Manejo de navegación con gamepad.

El uso del operador módulo permite que la selección sea circular. Si el usuario está en el primer elemento y presiona arriba, pasa al último. Si está en el último y presiona abajo, regresa al primero. Esta decisión evita que el usuario quede bloqueado en los extremos de la lista.

8.9. Selección de una ROM

Cuando el usuario presiona el botón de aceptar, el menú toma el juego seleccionado y construye su ruta completa. Después escribe esa ruta en el archivo temporal `/tmp/next_game.txt`. El script orquestador lee ese archivo y ejecuta Mednafen.

El Código 9 muestra este mecanismo.

```
1 if evento.type == pygame.JOYBUTTONDOWN:
2     if evento.button == 0 and juegos:
3         ruta_juego = os.path.join(ROM_DIR, juegos[indice_seleccionado])
4
5         with open("/tmp/next_game.txt", "w") as f:
6             f.write(ruta_juego)
7
8         pygame.quit()
9         sys.exit(0)
```

Código 9: Comunicación de la ROM seleccionada al orquestador.

La decisión de usar un archivo temporal simplifica la comunicación entre procesos. El menú no necesita ejecutar directamente el emulador ni mantenerlo como subprocesso. Su única responsabilidad es indicar qué ROM fue seleccionada.

8.10. Respuesta visual ante USB

La versión final del menú incorpora retroalimentación visual cuando se detecta una memoria USB. El objetivo no es que el usuario dependa de mensajes de terminal, sino que la propia interfaz indique lo que ocurre. El sistema puede mostrar una ventana emergente, reproducir una alerta y permitir la confirmación de la copia de ROMS.

Esta decisión fue importante porque el sistema debe operar como consola. En un dispositivo embebido, los mensajes relevantes deben presentarse en la interfaz del usuario. Si la copia USB se realizara únicamente en segundo plano, el usuario podría no saber si el sistema detectó la memoria

o si los juegos fueron agregados correctamente.

8.11. Actualización de la biblioteca

La interfaz revisa periódicamente el contenido de la carpeta de ROMS. Esto permite actualizar la lista cuando se agregan nuevos juegos. La decisión se relaciona directamente con la importación USB: después de copiar ROMS al almacenamiento local, el menú debe reflejar la nueva biblioteca.

Este comportamiento evita reiniciar manualmente la Raspberry Pi o cerrar la aplicación. Para el usuario, la consola simplemente actualiza su lista de juegos.

9. Integración con el sistema completo

Aunque este reporte se enfoca en la interfaz, el menú forma parte de un flujo mayor. El servicio de systemd inicia el script `start.sh`; dicho script ejecuta `menu.py`; el menú escribe la selección en `/tmp/next_game.txt`; y finalmente `start.sh` lanza Mednafen con la ROM seleccionada.

La integración general se resume en el Código 10.

```
retromenu.service
-> start.sh
    -> menu.py
        -> /tmp/next_game.txt
            -> mednafen ROM
                -> regreso al menu
```

Código 10: Flujo general de ejecución del sistema.

Esta separación modular permite explicar el sistema de forma ordenada. Si falla la interfaz, se revisa `menu.py`; si falla la ejecución del emulador, se revisa `start.sh`; si falla el arranque automático, se revisa el servicio `systemd`.

10. Resultados

El resultado de esta parte fue una interfaz gráfica funcional y coherente con el objetivo del proyecto. El menú permite operar la consola como un dispositivo dedicado, sin recurrir a un escritorio gráfico. La lista de juegos se genera automáticamente a partir del directorio local de ROMS, y la selección se realiza mediante gamepad.

Las pruebas principales se resumen en la Tabla 2.

Cuadro 2: Pruebas realizadas sobre la interfaz gráfica.

Elemento probado	Resultado observado
Pantalla completa	El menú se muestra como interfaz principal, sin escritorio ni ventanas adicionales.
Escaneo de ROMS	Se detectan archivos .nes, .sfc, .smc y .gba.
Formato de nombres	Los nombres se presentan de forma más legible al retirar extensiones y separadores.
Lista desplazable	El menú permite recorrer una biblioteca mayor a los elementos visibles en pantalla.
Gamepad	La selección se desplaza mediante el control y permite confirmar opciones.
Selección de juego	Se genera el archivo /tmp/next_game.txt con la ruta de la ROM seleccionada.
USB	La interfaz presenta mensajes visuales ante acciones relacionadas con importación.

El repositorio del proyecto se encuentra en:

<https://github.com/daniel13nic/proyecto-retro-embebidos>

El video demostrativo del prototipo se encuentra disponible en el siguiente enlace:

URL del video: https://youtu.be/Xch_X6m8AQw

11. Cuestionario

1. ¿Por qué la Raspberry Pi puede considerarse parte de un sistema embebido en este proyecto?
Porque fue configurada para realizar una función específica: operar como consola retro. No se utiliza como computadora de propósito general, sino como dispositivo dedicado con arranque automático, interfaz propia y periféricos definidos.
2. ¿Por qué se utilizó Pygame para la interfaz?
Porque permite dibujar una interfaz gráfica propia, operar en pantalla completa y procesar eventos de gamepad desde Python sin depender de un escritorio gráfico.
3. ¿Cuál es la función de `SDL_VIDEODRIVER=kmsdrm`?
Permitir que SDL y Pygame usen directamente la salida gráfica disponible, evitando depender de un entorno de escritorio.

4. ¿Cómo se filtran las ROMS disponibles?

El menú escanea `/home/pi/roms` y sólo acepta archivos con extensiones `.nes`, `.sfc`, `.smc` y `.gba`.

5. ¿Por qué se usa `/tmp/next_game.txt`?

Porque sirve como mecanismo simple de comunicación entre el menú gráfico y el script orquestador que ejecuta Mednafen.

6. ¿Qué ventaja tiene separar `menu.py` de `start.sh`?

Permite que la interfaz y la ejecución del emulador tengan responsabilidades distintas, facilitando pruebas, mantenimiento y depuración.

12. Conclusiones

La interfaz gráfica implementada permite que la Raspberry Pi funcione como una consola retro dedicada. El usuario interactúa con un menú en pantalla completa, controlado mediante gamepad, sin necesidad de usar escritorio gráfico, teclado o mouse durante la operación final.

En esta parte del proyecto se aplicaron conceptos de sistemas embebidos relacionados con entradas y salidas específicas, ejecución continua, restricciones de interfaz y separación de responsabilidades. La interfaz se encarga de mostrar información y recibir acciones del usuario, mientras que otros módulos del sistema se encargan de ejecutar el emulador, sincronizar ROMS y administrar el arranque.

Las decisiones tomadas en el desarrollo responden directamente a los requisitos del proyecto. Pygame se utilizó para construir una interfaz propia; kmsdrm se usó para evitar dependencia de escritorio; la lista desplazable permitió manejar una biblioteca de juegos mayor al espacio visible; el filtrado de extensiones evitó mostrar archivos inválidos; y el archivo temporal permitió comunicar la selección de ROM al orquestador de manera simple. Con ello, la interfaz cumple su función dentro de la solución embebida y permite operar el prototipo de forma directa.

Referencias

- [1] M. Barr, *Programming Embedded Systems in C and C++*. Sebastopol, CA, USA: O'Reilly, 1999.
- [2] Pygame, “Pygame documentation,” documentación oficial. Disponible en: <https://www.pygame.org/docs/>
- [3] Pygame, “pygame.display,” documentación oficial. Disponible en: <https://www.pygame.org/docs/ref/display.html>

- [4] Pygame, “pygame.joystick,” documentación oficial. Disponible en: <https://www.pygame.org/docs/ref/joystick.html>
- [5] Pygame, “pygame.event,” documentación oficial. Disponible en: <https://www.pygame.org/docs/ref/event.html>
- [6] Mednafen, “Mednafen multi-system emulator,” documentación oficial. Disponible en: <https://mednafen.github.io/>
- [7] D. N. Feregrino, L. E. Monter González y C. R. Ramírez Cervantes, “proyecto-retro-embebidos,” repositorio del proyecto. Disponible en: <https://github.com/daniel13nic/proyecto-retro-embebidos>
- [8] D. N. Feregrino, L. E. Monter González y C. R. Ramírez Cervantes, “Video demostrativo de consola retro embebida,” YouTube, 2026. Disponible en: https://youtu.be/Xch_X6m8AQw